

基本設計書

Jewelry Hunter (2D サイドアクションサンプル)

項目	内容
本書の目的	②要件定義書で定めた機能を、どのような仕組み・構成で実現するかを完成コードから逆算して整理したもの
版数	第 1.0 版

0. 本書の位置づけ

②要件定義書で定義した機能を、どのようなシーン構成・クラス構成・データの流れて実現するかを定義する。クラス単位の詳細な仕様（フィールド・メソッド）は④詳細設計書を参照する。

1. システム構成概要

本ゲームは「1 シーン=1 ステージ」という単位で Unity の SceneManagement クラスを利用して構成されている。各シーンは、次の 4 つの要素で構成される。

構成要素	役割
マネージャー系	GameManager クラス・TimeController クラスなどで、ステージ全体の進行・状態・スコア・UI 表示を管理する
プレイヤー	PlayerController クラスで入力に応じて移動・ジャンプし、他オブジェクトとの接触を判定する
ステージオブジェクト群	地形・敵 (EnemyController) ・アイテム (ScoreItem) ・ギミック (MovingBlock/SwitchAction/GimmickBlock/CannonController 等)
UI (Canvas)	スコア・残り時間の表示、クリア／オーバー時の結果パネル (RESTART/NEXT ボタン)

※ Title.unity と Result.unity は上記の「ステージ」構成を持たない専用シーンで、それぞれ TitleManager / ResultManager がシーン遷移のみを担当する。

2. 画面（シーン）遷移設計

ビルド設定に登録されたシーンの並び順は以下の通り（Stage 数が複数あると想定）

Title	Stage1	Stage2	StageO	StageO	Result
起動画面	→	→	→	→	最終結果

2.1 ステージ内の状態遷移（GameState 型の値）

- ・ InGame（プレイ中）→ ゴール到達で GameClear、失敗条件成立で GameOver に遷移する
- ※・ GameClear / GameOver → 各演出（画像・SE 表示、入力待ちの起動）の後、自動で GameEnd に遷移する

GameState は PlayerController の static フィールドとして持ち、GameManager が毎フレーム監視する構成になっている。

3. 主要システムの設計方針

◆ 3.1 プレイヤー移動・ジャンプ

- ・ 水平方向の入力値(axisH)を、そのまま Rigidbody2D.linearVelocity の X 成分に反映する「速度ベース」の移動方式とする。
- ・ 接地判定は、足元に向けて Physics2D.CircleCast（円形のレイキャスト）を行い、Ground レイヤーへの接触有無で判定する。
- ・ ジャンプは接地時のみ受け付け、入力があった場合はフラグを立てておき、FixedUpdate のタイミングで上方向へ瞬間的な力（Impulse）を加える。

◆ 3.2 カメラ制御

- ・ 通常時：プレイヤーの X/Y 座標を、ステージごとに設定した範囲（camLeft/Right/Top/Bottom）でクランプ（制限）した値をカメラ位置に反映する（追従 + 範囲制限）。
- ・ 強制スクロール時：プレイヤー操作によらず、一定速度でカメラ位置を加算し続ける（isForceScrollX/Y）。
- ・ 多重スクロール：背景オブジェクト（subScreen）は、カメラの X 座標の半分の速度で移動させ、遠近感（視差効果）を演出する。

◆ 3.3 状態管理（ゲームステート）

- ・ GameState 列挙型（InGame/GameClear/GameOver/GameEnd）を PlayerController の static フィ

ールドとして一元管理する。

- ・ GameManager は毎フレームこの状態を監視し、状態が変化した場合に UI 表示の切り替え・演出・サウンド再生・入力の有効/無効切り替えを行う。

◆ 3.4 スコア・制限時間管理

- ・ ステージ中の加点は、いったん PlayerController.score に一時保存される。GameManager が毎フレームこれを吸い上げて stageScore に積算し、0 にリセットする。
- ・ TimeController が制限時間をカウントダウンし、0 になったら GameManager がプレイヤーを強制的にゲームオーバーにする。
- ・ クリア時、残り時間を得点（1 秒 = 10 点）に変換したうえで、stageScore とあわせて totalScore（static 変数）へ積算する。static 変数のため、シーンをまたいでも値が保持される。

◆ 3.5 ギミック設計方針

- ・ 各ギミックは独立したコンポーネントとして実装し、Inspector 上の GameObject 参照によって疎結合に連携させる（例：SwitchAction は Inspector で対象の MovingBlock を参照し、Move()/Stop() メソッドを呼び出す）。
- ・ 各ギミックの動作パターン（移動距離・移動時間・待機時間・射程・発射間隔など）は Inspector で調整可能な public フィールドとして公開し、コードを変更せずにステージごとの難易度調整を可能にする。

◆ 3.6 UI 設計

- ・ Canvas 配下に、常時表示するスコア表示（ScoreBoard/ScoreText）・残り時間表示（TimeBar/TimeText）と、クリア／オーバー時のみ表示する結果パネル（Panel：画像 + RestartButton／NextButton）を配置する。
- ・ 結果パネル表示時は、結果に応じて RESTART ボタンまたは NEXT ボタンの一方だけを有効化（interactable）し、選べない選択肢を無効化する。

◆ 3.7 入力設計

- ・ Unity Input System の「Player」「UI」の 2 つのアクションマップを切り替えて使用する。
- ・ ゲームプレイ中は Player マップ（Move/Jump）を有効にし、結果表示中～次画面待ちの間は UI マップ（Submit）に切り替えて決定入力のみを受け付ける。

4. クラス構成（概念相関図）

クラス間の主な関係は次の通り（詳細なフィールド・メソッドは④詳細設計書を参照）。

起点クラス	関係	対象クラス
GameManager	⇒監視する	PlayerController.gameState (static)
GameManager	⇒更新する	TimeController → UI (TimeText) / UI (ScoreText)
PlayerController	⇒接触時に取得する	ScoreItem → ItemData (得点・名称・画像)
SwitchAction	⇒操作する	MovingBlock (Move / Stop)
CannonController	⇒生成する	ShellController (砲弾)

5. 共通仕様

5.1 タグ

タグ	用途
Goal	ゴール地点。プレイヤーが接触するとゲームクリア
Dead	死亡判定エリア。プレイヤーが接触するとゲームオーバー
ScoreItem	スコアアイテム。プレイヤーが接触すると加点して消える
Switch	スイッチギミック（EnemyControllerの反転判定などにも利用）
Player	プレイヤー本体（各ギミックからの参照・判定に利用）

5.2 レイヤー

レイヤー	用途
Ground	接地判定（CircleCast）の対象
Cannon	大砲オブジェクト用
Shell	大砲が発射する砲弾用